

The Stochastic $O(nm)$ Wu Pipeline

Simulation_code_MCREML_pooled_matfree_StochasticWu — matfree_new with exactly one block replaced: the matrix-free pooled W apply, swapped for the stochastic operator of *Different way for REML*, section 2

Summary. This pipeline is Simulation_code_MCREML_pooled_matfree_new verbatim with a single block replaced — the # matrix-free pooled W apply section. The model, the dense simulation kernel, the phenotype, CG, SLQ, the AI-REML update and the four-step SLURM chain are all unchanged, so the phenotypes are the **same data** matfree_new is run on and a run here is directly comparable to a matfree_new run. The exact apply costs $O(nm^2/G)$; the replacement costs $O(nmN_w)$ with N_w frozen Rademacher probes. The catch is that the $O(nm)$ collapse only exists after the $1/\sigma_{ab}$ pair scaling is dropped, so the estimator now targets a **slightly different kernel** than the one the data came from — a deterministic bias that no N_w removes. Sections 2–6 derive the operator and cost it, 7 covers the REML solver, 8 the SLURM chain, and 9–11 the verification (with measured numbers), the diagnostics and the knobs.

1 What is different from matfree_new, and why

matfree_new applies W exactly, contracting through the per-gene $m_g \times m_g$ weight matrices V_g, R_g, T_g that encode the pair standardization $h_{ab} = (Z_a \circ Z_b - \mu_{ab} \mathbf{1})/\sigma_{ab}$. The weight $B_{ab} = 1/\sigma_{ab}^2$ sits inside the quartic sum $\sum_{a \neq b} B_{ab} Z_{ta} Z_{tb} Z_{sa} Z_{sb}$ and is full rank, which is precisely what forces $O(nm^2)$.

Section 2 of the note removes it. With the **centering** kept but not the scaling, $h_{ab} = Z_a \circ Z_b - \mu_{ab} \mathbf{1}$, the quartic collapses onto the Hadamard square of the additive GRM (Section 3) and an $O(nm)$ apply becomes possible. The note calls the result “the **std** linear operator for Wu ”; **std** there means the centering is carried (the v_R, s_T corrections), **not** the $1/\sigma_{ab}$ scaling.

1.1 The estimand gap this opens

The simulation is matfree_new’s and builds W_{std} from **standardized** pair columns. The operator estimates W_{cen} , the **centered-but-unscaled** kernel. These are not the same matrix, so the estimator carries a **deterministic kernel bias** on top of its Monte-Carlo error, and **no N_w makes it smaller**.

This is a deliberate trade. Keeping the simulation identical means the phenotypes are the same data matfree_new runs on, so the comparison isolates the estimator. The alternative — simulating from W_{cen} too — would measure only the stochastic error but would no longer be comparable to matfree_new at all.

The gap is governed by $\sigma_{ab}^2 = 1 + 2r_{ab}^2$. Under $\text{MODE}=\text{RandomSNP}$ $r_{ab}^2 = O(1/n)$, so the two kernels agree to $O(1/n)$: $\text{tr}(W_{\text{std}})/n = 1$ exactly against $\text{tr}(W_{\text{cen}})/n = 1 + 2r^2$. Measured at $n = 200$, $m = 60$, $G = 3$ (check C0):

Quantity	Measured
$\text{tr}(W_{\text{std}})/n$	1.000000
$\text{tr}(W_{\text{cen}})/n$	0.996564
$\ W_{\text{cen}} - W_{\text{std}}\ _F / \ W_{\text{std}}\ _F$	0.0687

Under strong LD the two are not on the same scale and results are not comparable at all. `verify_stochastic_wu.py` reports this number on any genotype and separates it from the Monte-Carlo error – run it before trusting a new `MODE`.

2 The model

Let $Z \in \mathbb{R}^{n \times m}$ be the column-standardized dosage matrix (`additive_design`: each column mean 0, variance 1 with `ddof=0`). The m SNPs are cut into G **contiguous** gene blocks by `np.array_split`, so gene g owns $Z_g \in \mathbb{R}^{n \times m_g}$. With no fixed effects (y mean-centred),

$$y = g_{\text{xxg}} + e, \quad V = \text{Var}(y) = \sigma_{\text{gxxg}}^2 W + \sigma_e^2 I, \quad \theta = (\sigma_{\text{gxxg}}^2, \sigma_e^2).$$

Only pairs **inside** a gene contribute, and every such pair gets equal weight – the division is by the **global** pair total, once:

$$W = \frac{1}{P} \sum_{g=1}^G H_g H_g^T = \frac{1}{P} \sum_{g=1}^G \sum_{a < b \in g} h_{ab} h_{ab}^T, \quad P = \sum_{g=1}^G \binom{m_g}{2}.$$

A gene with $m_g < 2$ contributes no pair and is skipped everywhere. Two structural facts hold for **both** kernels and are used constantly below:

1. **$W\mathbf{1} = \mathbf{0}$ exactly.** Every h_{ab} is mean-centred, so $h_{ab}^T \mathbf{1} = 0$. Hence $\lambda_{\min}(W) = 0$ and $\lambda_{\min}(V) = \sigma_e^2$ exactly.
2. **$W \succeq \mathbf{0}$** , being a sum of outer products.

`build_w_pooled` forms W_{std} densely in pair-batches ($O(n^2 P)$ time, $O(n^2)$ memory), masking degenerate pairs ($\sigma_{ab} \leq 10^{-10}$) to zero. **It is used by the simulation only**, because a Cholesky factor needs an explicit matrix. The estimator never touches it.

3 The collapse: from a pair sum to a Hadamard square

Start unstandardized and un-centred, one gene, and expand the pair sum over **ordered** pairs $a \neq b$ (each unordered pair counted twice, hence $1/(2p)$):

$$2p W_{ts}^{\text{raw}} = \sum_{a \neq b} Z_{ta} Z_{tb} Z_{sa} Z_{sb} = \left(\sum_a Z_{ta} Z_{sa} \right)^2 - \sum_a Z_{ta}^2 Z_{sa}^2.$$

In matrix form, with $K_w = ZZ^T$ (the additive GRM, $n \times n$, **never formed**) and $D = Z \circ Z \in \mathbb{R}^{n \times m}$,

$$W = \frac{1}{2p} ((ZZ^T) \circ (ZZ^T) - (Z \circ Z)(Z \circ Z)^T) = \frac{1}{2p} (K_w \circ K_w - DD^T).$$

The second piece is free: $DD^T u = D(D^T u)$ costs $O(nm)$ and D costs the same to build and store as Z . All the difficulty is in $K_w \circ K_w$, whose exact apply $\text{dg}(Z(Z^T \text{diag}(u)Z)Z^T)$ is the $O(nm^2)$ route (see `KKu.typ`). A full-rank weight inserted between the two contractions – which is what $1/\sigma_{ab}^2$ is – blocks the identity above entirely.

4 The stochastic apply

4.1 The identity

For any A, B and any zero-mean, unit-variance probe v (the pipeline uses Rademacher, $v_i \in \{\pm 1\}$),

$$(A \circ B)u = \mathbb{E}[(A \text{diag}(u)B^T v) \circ v].$$

The reason is one line: entry t of the right side is $\mathbb{E}[\sum_{j,s} A_{tj} u_j B_{sj} v_s v_t]$, and $\mathbb{E}[v_s v_t] = \delta_{st}$ leaves $\sum_j A_{tj} B_{tj} u_j$. Applying it with $A = B = K_w$ and truncating the expectation at N_w probes $V = [v_1, \dots, v_{N_w}] \in \mathbb{R}^{n \times N_w}$:

$$(K_w \circ K_w)u \approx \frac{1}{N_w} \sum_{i=1}^{N_w} (K_w \text{ diag}(u) K_w v_i) \circ v_i = \frac{1}{N_w} ((Z(Z^T(u \circ (Z(Z^T V)))) \circ V) \mathbf{1}_{N_w}).$$

Read right to left: $Y = Z(Z^T V) = K_w V$, then scale rows by u , then apply K_w again, then contract against V . **Y does not depend on u** , so it is built once at setup and every apply is two gemms against Z .

4.2 Restoring the centering

Centering h_{ab} subtracts $\mu_{ab} \mathbf{1}$, which contributes three further terms. The full operator splits as

$$Wu = \underbrace{\frac{1}{2p} [(K_w \circ K_w)u - D(D^T u)]}_{\text{raw}} + \underbrace{\frac{1}{2p} [-s_u v_R + (s_u s_T - u^T v_R) \mathbf{1}]}_{\text{centering}}, \quad s_u = \mathbf{1}^T u,$$

with $v_R = \text{dg}(ZZZ^T)$, $R = \mu - I$, $\mu = Z^T Z/n$, and $s_T = \mathbf{1}^T (R \circ R) \mathbf{1}$. Both correction quantities are u -independent, so they are **setup**, not apply. And both are just the raw bracket evaluated at $u = \mathbf{1}$, so the same stochastic estimator delivers them — no new machinery:

$$\hat{v}_R = \frac{1}{nN_w} ((Z(Z^T(Z(Z^T V)))) \circ V) \mathbf{1}_{N_w} - (Z \circ Z) \mathbf{1}_m, \quad \hat{s}_T = \frac{1}{n} \mathbf{1}^T \hat{v}_R.$$

The second identity is exact, not an estimate: $\mathbf{1}^T \text{dg}(ZZZ^T) = \text{tr}(RZ^T Z) = n \sum_{a \neq b} \mu_{ab}^2 = n s_T$. So s_T costs nothing once v_R is in hand — and Section 5.3 shows this is also what makes $\widehat{W} \mathbf{1} = 0$ hold identically.

4.3 The pooled operator: each gene computed whole

The note groups the pooled operator as “all the raw brackets, then one pooled centering”. The code does **not**. The centering is **linear** in v_R and s_T , and both pool by plain summation over genes, so it distributes into the gene sum:

$$\widehat{W}U = \frac{1}{2P} \sum_{g=1}^G [\text{raw}_g(U) - v_{Rg} s_U^T + \mathbf{1}(s_{Tg} s_U - U^T v_{Rg})^T] = \frac{1}{2P} \sum_{g=1}^G 2S_g U,$$

$$\text{raw}_g(U) \approx \frac{1}{N_w} \sum_{i=1}^{N_w} v_{gi} \circ (Z_g(Z_g^T(U \circ y_{gi}))) - D_g(D_g^T U), \quad y_{gi} = K_{w,g} v_{gi},$$

with $S_g = \sum_{a < b \in g} h_{ab} h_{ab}^T$ and $s_U = U^T \mathbf{1}$.

So `_gene_apply` returns a gene’s **complete** contribution — quartic, DD^T and that gene’s own centering together — and `compute_WU_pooled` is only the gene sum and the global $1/(2P)$. The per-pair weight P is the one quantity that genuinely does not decompose by gene, because the Pooled Model divides once by the **global** pair total.

This is not merely tidier. Three things follow:

1. **$\widehat{W}_g \mathbf{1} = 0$ holds per gene**, not only after pooling (Section 5.3’s cancellation is a per-gene identity), so a fault in one gene shows up instead of being averaged into the sum. `verify_formula_match.py` check F2b prints it gene by gene.

2. **A gene's apply is checkable in isolation** against its own dense S_g .
3. **The cost is unchanged:** G rank-one updates instead of one, $O(Gnc)$ against the $O(nmN_w c)$ quartic.

Each gene draws its **own** probe block V_g , and keeps its own $v_{Rg}, s_{Tg} - G$ vectors of length n instead of one, about 80 KB at $n = 1000, G = 10$. In code this is `setup_pooled` (`setup`) and `compute_WU_pooled` (`apply`) — deliberately the **same function names** `matfree_new` uses, so the swap is literal and the call sites in `mc_reml` are unchanged apart from the state object. `verify_formula_match.py` checks the code against a literal, unbatched, one-gene-one-column transcription of the note's **own** grouping and agrees to 4×10^{-16} — which is simultaneously the proof that the two groupings coincide.

5 Three things the code adds beyond the formula

The formula as written is not usable inside REML. Three modifications are required, and each one is load-bearing.

5.1 The probes are frozen

They are drawn **once**, from a seeded generator, at setup, and reused for the whole run. Redrawing inside an apply would make $\widehat{W}$ a **different matrix on every call**: CG would descend on a moving target and Lanczos would build a basis for no operator at all. Frozen, $\widehat{W}$ is a genuine deterministic matrix and REML is a deterministic function of it given `w_seed`.

5.2 The apply is symmetrized

At finite N_w the estimator's matrix is

$$\hat{A}_{ts} = \frac{1}{N_w} \sum_{i=1}^{N_w} v_i(t) K_{w,ts} (K_w v_i)_s,$$

which is symmetric only **in expectation**. CG and Lanczos both require symmetry, so the apply is averaged with its transpose. The transpose is the same expression with V and Y swapped, $\hat{A}^T u = N_w^{-1} \sum_i y_i \circ (Z(Z^T(v_i \circ u)))$, so symmetrizing doubles the work and changes nothing else. `symmetrize=False` recovers the note's formula literally and exists only for verification.

Measured at $N_w = 200$ (check C3): relative asymmetry 4.7×10^{-1} for the literal operator against 1.8×10^{-16} symmetrized — and, decisively, $\lambda_{\min}/\lambda_{\max} = -0.1295$ literal against -0.0000 symmetrized. The literal operator is **badly** indefinite; the symmetrized one is not.

5.3 $\widehat{W}\mathbf{1} = 0$ holds exactly, on every draw

This is not an approximation that improves with N_w ; it is an algebraic identity, and it is easy to lose. At $u = \mathbf{1}$:

- the quartic estimate becomes exactly $n \cdot (\hat{v}_R + D\mathbf{1}_m)$, since the probes and the K_w -applies are the same ones $\hat{v}_R$ was built from;
- $D(D^T \mathbf{1}_n) = n D\mathbf{1}_m$, because column standardization with `ddof=0` gives $\sum_t Z_{ta}^2 = n$ for every SNP;
- so the raw bracket is exactly $n\hat{v}_R$;
- the correction is $-n\hat{v}_R + (n\hat{s}_T - \mathbf{1}^T \hat{v}_R)\mathbf{1}$, and the parenthesis vanishes **identically** because $\hat{s}_T = \mathbf{1}^T \hat{v}_R / n$.

The two cancel. This holds **only while** $\hat{v}_R$ is built by the same estimator the apply uses, which is why `setup_pooled` symmetrizes $\hat{v}_R$ in step with the quartic. The transpose at $u = \mathbf{1}$

is $N_w^{-1} \sum_i (K_w v_i) \circ (K_w v_i) = (Y \circ Y) \mathbf{1} / N_w$, already in hand, so this costs no extra gemm and stays unbiased. Measured: $\|\widehat{W} \mathbf{1}\| \approx 6 \times 10^{-14}$ for both variants — machine precision, not a small residual.

5.4 Unbiasedness

$\mathbb{E}[\widehat{W}] = W_{\text{cen}}$ exactly. Everything above is linear in $\hat{v}_R$ and $\hat{s}_T$, and each piece is unbiased on its own ($\mathbb{E}[v_t(K_w \text{diag}(u)K_w v)_t] = \sum_s K_{w,ts}^2 u_s$ for Rademacher probes), so reusing one probe set for the quartic **and** for $\hat{v}_R$ costs variance, not bias. Check C4 confirms a clean $1/\sqrt{R}$ decay over R independent draws with no floor: $0.245 \rightarrow 0.072 \rightarrow 0.024 \rightarrow 0.008$ for $R = 10, 10^2, 10^3, 10^4$, a factor 3.16 per decade.

This does not make REML unbiased, on two counts: $\hat{h}^2$ is a strongly nonlinear function of $\widehat{W}$, and $W_{\text{cen}} \neq W_{\text{std}}$ (Section 1.1).

6 Implementation and cost

6.1 Blocking

Setup, per gene: keep Z_g , $D_g = Z_g \circ Z_g$, the frozen probes V_g ($n \times N_w$), $Y_g = K_{w,g} V_g$, and the gene's own v_{Rg} , s_{Tg} . Nothing $m \times m$ is ever formed — unlike `matfree_new`, whose setup has an m^2 term from V_g, R_g, T_g and whose state keeps V_g live for the whole run.

Apply, per gene: right-hand sides run in blocks of c_b columns sized so the lifted $(n, N_w c_b)$ buffer stays near `buf_elems` = 4×10^6 doubles, so each block is two (or four, symmetrized) gemms plus one einsum contraction.

```
Tb = (Y[:, :, None] * Ub[:, None, :]).reshape(n, Nw * w) # u .* Y, lifted
Ob = (Zg @ (Zg.T @ Tb)).reshape(n, Nw, w) # K applied
acc = np.einsum('ni,niw->nw', Vp, Ob) # contract vs V
if symmetrize: # transpose: V <-> Y
    Tb = (Vp[:, :, None] * Ub[:, None, :]).reshape(n, Nw * w)
    Ob = (Zg @ (Zg.T @ Tb)).reshape(n, Nw, w)
    acc = 0.5 * (acc + np.einsum('ni,niw->nw', Y, Ob))
out[:, s:e] = acc / Nw
out -= Dg @ (Dg.T @ U) # a = b terms; exact, 0(n m_g)
s_U = U.sum(axis=0) # this gene's own centering:
out -= np.outer(vRg, s_U) # two rank-one updates, 0(n c)
out += np.outer(np.ones(n), sTg * s_U - U.T @ vRg)
```

The last three lines are what makes the gene **whole**: `_gene_apply` returns $2S_g U$, not a partial sum awaiting a pooled correction.

6.2 Cost

Object	This pipeline	<code>matfree_new</code>
Setup (all genes)	$8nmN_w$	$O(nm^2/G + m^2/G)$
Apply, per RHS, gene g	$8nm_g N_w$	$2nm_g^2$
Memory	$O(nm + GnN_w)$	$O\left(nm + \sum_g m_g^2\right)$
Dense W	never formed	never formed

So the stochastic route is cheaper **only when** $4N_w < m_g$. Accuracy runs the other way: the relative error of $\widehat{W}U$ against W_{cen} is

$$\|\widehat{W}U - W_{\text{cen}}U\|/\|W_{\text{cen}}U\| \approx 0.3\sqrt{n/N_w},$$

flat in m . Check C1 at $n = 200$: 0.890, 0.634, 0.423, 0.285, 0.214, 0.148 for $N_w = 25 \dots 800$ (a factor $\sqrt{2}$ per doubling), and at fixed $N_w = 200$ the error grows 0.188, 0.285, 0.512, 0.798 as n runs 100 $\rightarrow$ 800 (a factor $\sqrt{2}$ per doubling of n). Holding accuracy fixed therefore needs $N_w \sim n$.

At the shipped defaults `matfree_new` wins on both counts. With $m = 1000$, $G = 10$ so $m_g = 100$, and $N_w = 200$: the cost ratio is $4N_w/m_g = 8$, i.e. the stochastic apply is $8 \times$ **more** expensive, while its relative error is $0.3\sqrt{1000/200} \approx 0.67$. This is not a misconfiguration — charting exactly this trade-off is what the N_w sweep is for. The regime the operator was built for is m_g in the thousands with n small, which the within-gene decomposition does not produce.

7 The estimator

Unchanged from `matfree_new` except that Wu comes from Section 4 — and one adaptation in the SLQ guard, which is the **only** place outside the W apply where `Function_MCREML.py` departs from `matfree_new`.

7.1 V^{-1} by batched CG

$VU = \sigma_{\text{gsg}}^2 \widehat{W}U + \sigma_e^2 U$ is the only primitive. `_cg_batched` solves all c right-hand sides together with per-column CG scalars, so one V -pass advances every column, and warm-starts from the previous REML iteration's solution.

7.2 Score traces by stochastic Lanczos quadrature

V is **affine** in $\widehat{W}$, so a Lanczos run on $\widehat{W}$ — not on V — gives nodes μ_j and weights τ_j valid at **every** variance setting the optimizer visits. With N_{mc} Rademacher probes u_l and k Lanczos steps (`slq_k = 25`, full reorthogonalization applied twice):

$$\text{tr}(V^{-1}I) \approx \text{mean}_l \|u_l\|^2 \sum_j \tau_j / (\sigma_{\text{gsg}}^2 \mu_j + \sigma_e^2),$$

$$\text{tr}(V^{-1}W) \approx \text{mean}_l \|u_l\|^2 \sum_j \tau_j \mu_j / (\sigma_{\text{gsg}}^2 \mu_j + \sigma_e^2).$$

Per REML iteration this is $O(N_{\text{mc}}k)$ arithmetic — **no solve, no apply**. The entire spectral cost of a run is the one Lanczos pass: k applies of width N_{mc} .

The one forced adaptation. `slq_reliable` guards the cached rule with the Gauss/CG bound $2\rho^{2k} \leq \text{tol}$, $\rho = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$, and needs $\lambda_{\min}(V)$ to form κ . `matfree_new` could take $\lambda_{\min}(W) = 0$ for granted, since its W is PSD, giving $\lambda_{\min}(V) = \sigma_e^2$.

$\widehat{W}$ is PSD only **in expectation**. It satisfies $\widehat{W}\mathbf{1} = 0$ exactly, so 0 is always an eigenvalue, but at finite N_w it can dip **negative elsewhere**, and then $\lambda_{\min}(V) = \sigma_e^2 + \sigma_{\text{gsg}}^2 \lambda_{\min}(\widehat{W}) < \sigma_e^2$. If that goes non-positive, V is indefinite, CG stops meaning anything, and REML returns a confident meaningless answer (historically $\hat{h}^2 = 1$). So `slq_setup` carries the smallest **unclipped** Ritz value through and `slq_reliable` conditions on it, returning `False` — falling back to exact probe solves — when the lower end is non-positive. Passing `w_min = 0.0` (the default) reproduces `matfree_new` exactly. Nodes are still clipped at 0 afterwards, so $\sigma_{\text{gsg}}^2 \mu_j + \sigma_e^2$ can never be driven non-positive.

7.3 The damped AI-Newton step

With $x = V^{-1}y$, $K_1 = W$, $K_2 = I$:

$$s_i = \frac{1}{2}(x^T K_i x - \text{tr}(V^{-1} K_i)), \quad \mathcal{A}_{ij} = \frac{1}{2}(K_i x)^T V^{-1}(K_j x), \quad \theta \leftarrow \theta + (\mathcal{A} + \lambda I)^{-1} s.$$

W is often nearly collinear with I , so $\mathcal{A}$ is near-singular and an undamped step explodes. Three stabilizers, all of which leave a well-identified fit essentially unperturbed: a Levenberg–Marquardt ridge $\lambda = 10^{-3} \cdot \text{diag } \mathcal{A}$, a trust region $\max_i |\Delta_i| \leq 0.5 \cdot \text{Var}(y)$, and a box clamp $\sigma_i^2 \in [10^{-9}, 5 \cdot \text{Var}(y)]$.

7.4 Where the applies go

Per replicate, at the shipped defaults ($N_{\text{mc}} = 100$, $k = 25$, 30 iterations):

Stage	Width	Column-applies
Operator setup (Y , $K_w Y$, $\hat{v}_R$)	N_w	$2N_w$ equivalent
SLQ Lanczos (once)	N_{mc}	$k \cdot N_{\text{mc}} = 2500$
CG for $x = V^{-1}y$, per iteration	1	$\approx$ CG iters
Wx , per iteration	1	1
CG for $\mathcal{A}$, per iteration	2	$2 \cdot$ CG iters
SLQ-guard fallback, if triggered	N_{mc}	$N_{\text{mc}} \cdot$ CG iters

8 The four-step SLURM chain

MCREML_pipeline.sh submits four dependent jobs, exactly as matfree_new does. Two tags separate data from estimate:

Tag	Value
TAG (data)	{MODE}_s2g _g {S2GXG}_s2e{S2E}_n{N}_m{M}_G{G}
FILENAME (estimate)	{MODE}_s2g _g {S2GXG}_s2e{S2E}_n{N}_m{M}_G{G}_Nw{NW}

TAG carries no N_w ; FILENAME does. That is the one naming change from matfree_new, and it is what lets an accuracy-vs- N_w sweep coexist under one set of phenotypes instead of overwriting itself.

Step	Job	What it does	Array
1	Cholesky.sh	build_w_pooled densely (standardized pair columns), factor $LL^T = \sigma_{\text{g} \times \text{g}}^2 W + 10^{-10} I$, save Lg _g _g_{TAG}.npy	single
2	Phenotype.sh	$y = Lu_1 + \sqrt{\sigma_e^2} u_2$, each component rescaled to its exact target variance, then mean-centred	1–300
3	MCREML.sh	reads the genotype only ; MC_REML with the Section 4 operator (N_w frozen probes) and SLQ traces	1–300
4	combine_code.sh	concatenate per-rep files, average timings, delete the per-rep directories	single

Sampling-error removal. simulate_remove_sampling_err rescales $g_{\text{g} \times \text{g}}$ and e to hit $\sigma_{\text{g} \times \text{g}}^2$ and σ_e^2 exactly in-sample, so the spread across the 300 replicates reflects estimator variance, not draw variance.

Seeds. seed=rep fixes the **trace** probes; w_seed fixes the **operator** probes. W_SEED=-1 in the driver means “use 10000 + rep”, so the operator draw is resampled across replicates and its randomness shows up in the 300-rep spread. Pin W_SEED ≥ 0 to hold **one** operator draw fixed instead.

Two probe counts, not one. NMC = trace probes (how well $\text{tr}(V^{-1}K_i)$ is estimated). NW = frozen operator probes (how close $\widehat{W}$ is to W_{cen}). They are independent, and NW is the one that governs Section 6’s error.

Cleanup differs from matfree_new in one respect. matfree_new’s step 4 deletes \$LGXG_FILE and \$PHENO_DIR; here they are **kept**, because they are keyed by TAG and re-drawing them between N_w settings would confound the sweep with phenotype noise. Delete them by hand once the sweep is finished.

Outputs.

Path (under \$DIR)	Contents
result/{FILENAME}.txt	one (s2gxcg,s2e) per rep — read by calc_stats.py (mean, median, sd, 95% CI)
time/result/timing_{FILENAME}.txt	mean estimation wall-clock
time/result/W_timing_*.txt	dense- W build time — simulation-only cost, no N_w in the name

9 Verification

Two scripts, both run from the pipeline directory with no SLURM. All numbers below are from an actual run at $n = 200$, $m = 60$, $G = 3$.

verify_formula_match.py — **is the code the note’s formula?**

- **F1** single gene: the formula transcribed literally (nothing hoisted, nothing batched, no symmetrization, one column at a time) vs the pipeline driven by the same frozen probes. Probes identical, Y identical, v_R to 5×10^{-14} , s_T to 5×10^{-15} , Wu to 4.5×10^{-16} .
- **F2** pooled, in the **note’s** grouping (all raw brackets, then one pooled centering) against the code’s (each gene whole) — 3.9×10^{-16} . This is the check that the regrouping of Section 4.3 is exact.
- **F2b** the per-gene identity $\widehat{W}_g \mathbf{1} = 0$, printed gene by gene: $\approx 2 \times 10^{-11}$ unnormalized per gene, 1.5×10^{-13} pooled.
- **F3** quantifies the one intentional deviation, symmetrization (0.31 relative, as expected at $N_w = 60$), and confirms $\widehat{W} \mathbf{1} = 0$ to 4×10^{-14} for both variants.

verify_stochastic_Wu.py — **is the operator correct as a matrix?** Its job is to keep the **two** error sources apart:

- **C0** the kernel gap (Section 1.1): 0.0687, deterministic.
- **C1** $\widehat{W}$ vs W_{cen} : decays as $N_w^{-1/2}$, grows as $\sqrt{n}$, flat in m . This is the error the operator controls.
- **C2** $\widehat{W}$ vs W_{std} — what the phenotype was actually drawn from. Same decay, but it **flattens onto the C0 gap**: $0.892 \rightarrow 0.100$ over $N_w = 25 \dots 3200$, against a floor of 0.0659. The contrast between C1 and C2 is the whole point of the script.
- **C3** the frozen operator as a matrix — asymmetry, $\|\widehat{W} \mathbf{1}\|$, $\lambda_{\min}/\lambda_{\max}$, tr/n — for the literal, the symmetrized, and **both** exact kernels side by side.
- **C4** unbiasedness toward W_{cen} : clean $1/\sqrt{R}$, no floor.
- **C5** end-to-end REML on 5 replicates across $N_w \in \{200, 400, 800, 3200\}$.

What C5 shows, and what it does not. At true $h^2 = 0.2$ the five-replicate means are 0.281, 0.243, 0.324, 0.331 for $N_w = 200, 400, 800, 3200$, with per- N_w standard deviations near 0.11. So the estimate does **not** converge on the truth as N_w grows — consistent with the C0 kernel gap surviving the $N_w \rightarrow \infty$ limit. But five replicates at $sd \approx 0.11$ gives a standard error near 0.05, which is far too coarse to separate the kernel bias from ordinary small-sample REML noise on a weakly-identified problem. Treat C5 as a smoke test that the estimator runs and lands in the right region, **not** as a bias measurement. The 300-replicate pipeline is what measures bias.

10 Reading a run

There is no separate diagnostics file — the pipeline writes exactly what `matfree_new` writes. What to look at:

calc_stats.py result/{FILENAME}.txt. Mean, median, sd and 95% CI of $\hat{\sigma}_{\text{gxg}}^2$ and $\hat{\sigma}_e^2$ over the 300 replicates. Compare against the `matfree_new` run at the same TAG: the phenotypes are identical, so the difference is the operator, kernel gap included.

The verbose line (set `verbose=True` on `MC_REML` for a single interactive run). It prints the SLQ fallback count and the smallest unclipped Ritz value of $\widehat{W}$. That Ritz value is the number that matters: 0 for an exact PSD kernel, slightly negative for $\widehat{W}$, and the more negative it goes the closer $V = \sigma_{\text{gxg}}^2 \widehat{W} + \sigma_e^2 I$ is to losing positive definiteness — at which point CG stops meaning anything.

A large fallback count means either `slq_k` is too small or the fit has drifted onto the $\sigma_e^2 \rightarrow 0$ boundary. Check which before raising k .

Before trusting a new MODE, run `verify_stochastic_Wu.py` on that genotype and read C0. Under strong LD the kernel gap stops being $O(1/n)$ and the two pipelines' σ_{gxg}^2 are no longer on the same scale.

11 Knobs

Knob	Default	Effect
NW	200	frozen operator probes; error $\approx 0.3\sqrt{n/N_w}$, cost linear in N_w . The sweep variable.
NMC	100	trace probes; independent of NW
SLQ_K	25	Lanczos steps; raise if the fallback count is large
TRACE_METHOD	slq	hutchinson pays N_{mc} CG solves per iteration instead
W_SEED	-1	-1: 10000 + rep. ≥ 0 : one fixed draw
G	10	gene count; sets $m_g = m/G$, which sets the crossover $4N_w < m_g$
symmetrize	True	False = the note's literal asymmetric operator; verification only

To sweep N_w : edit NW, comment out steps 1–2, and resubmit steps 3–4 against the existing LGXG_FILE and PHENO_DIR. The phenotypes do not depend on N_w , and step 4 no longer deletes them.